2026 IEEE International Symposium on High-Performance Computer Architecture (HPCA)

LILo: Harnessing the On-chip Accelerators in Intel CPUs for Compressed LLM Inference Acceleration

Hyungyo Kim[1]*, Qirong Xia[1]*, Jinghan Huang[1], Nachuan Wang[1], Younjoo Lee[2],
Jung Ho Ahn[2], Wajdi K Feghali[3], Ren Wang[3], and Nam Sung Kim[1],
[1]University of Illinois at Urbana-Champaign, [2]Seoul National University, and [3]Intel Corporation
{hyungyo2, qirongx2, jinghan4, nachuan3, nskim}@illinois.edu, {younjoo0614, gajh}@snu.ac.kr,
{wajdi.k.feghali, ren.wang}@intel.com

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408577

***Abstract*—The ever-growing sizes of large language models (LLMs) introduce significant infrastructure challenges due to their immense memory capacity demands. While the de facto approach has been to deploy multiple high-end GPUs, each with a limited memory capacity, the prohibitive cost of such systems has become a major barrier to the widespread deployment of frontier LLMs. As a result, CPU-based inference has become an appealing and cost-efficient alternative, since a CPU can offer an order of magnitude larger memory capacity at a fraction of the cost while providing competitive throughput for matrix-vector multiplication with the latest Advanced Matrix Extensions (AMX). It not only broadens accessibility for users without multi-GPU setups but also enables hyperscalers to leverage underutilized CPU servers to accommodate temporarily surging inference demand. Nevertheless, even CPU's large memory capacity has become insufficient to serve LLMs with hundreds of billions of parameters. Under the memory capacity constraint, we may offload parameters to storage devices and fetch them on demand, but doing so significantly degrades inference performance due to the high latency and low bandwidth of storage devices.**

To address this challenge, we propose LILo, an LLM inference framework that leverages In-memory Analytics Accelerator (IAA) in the latest Intel CPUs, to accelerate inference under memory capacity constraints. By storing model parameters in a compressed format and decompressing them on demand using IAA, LILo enables significantly reduced storage access during inference under memory capacity constraints while preserving the model accuracy and behavior. LILo orchestrates the concurrent execution of on-chip accelerators, *i.e.,* IAA, Advanced Vector Extensions (AVX), and AMX, to facilitate high-throughput decompression alongside inference computation. Furthermore, LILo implements selective compression, a Mixture-of-Expert (MoE)-aware optimization that reduces the decompression overhead by up to 1.9×. We demonstrate that LILo reduces inference latency by up to 4.9× and 4.3× for Llama3-405B and DeepSeek-R1, respectively, under memory capacity constraints compared to the baseline inference solely relying on storage-offloading without compression.

I. Introduction

The remarkable success of large language models (LLMs) in recent years has been largely driven by scaling model size, a trend captured by the empirical scaling law [37]. Frontier models like GPT-4 [44], Llama3-405B [32], and DeepSeek-R1 [26] scale to hundreds of billions of parameters to deliver exceptional accuracy across a wide range of tasks. As a result, deploying these models requires multiple GPUs, since even expensive GPUs are designed with a limited memory capacity to provide higher memory bandwidth, exemplifying a fundamental trade-off in memory technology. For example, Llama3-405B model [32] requires 2 DGX-H100 instances, each with 8 GPUs, to simply provide enough memory capacity to store the 405B parameters in BF16 format (*i.e.,* 754 GB), costing roughly $800K [12]. Such prohibitively expensive infrastructure restricts access to state-of-the-art LLMs and motivates more affordable deployment alternatives.

In this context, CPU-based inference has emerged as a practical, low-cost alternative—appealing not only to users or organizations lacking access to multi-GPU systems but also to hyperscalers seeking to exploit underutilized CPU servers for temporarily surging inference demand [21], [46]. The memory capacity requirement for serving recent large models, nonetheless, can be challenging to meet even with CPUs due to limited DRAM slots, platform constraints, power or thermal limits, or budget constraints. To enable the deployment of large models that exceed the CPU's memory capacity, recent frameworks, including HuggingFace Accelerate [33] and DeepSpeed Zero-Inference [19], support offloading model parameters to storage devices (SSD/HDD). However, the low bandwidth and high latency of storage access significantly degrade inference performance.

Quantization has been the mainstream approach for reducing model size. Yet, it introduces several critical drawbacks, including potential degradation in model accuracy [43], increased vulnerability to adversarial attacks [29], unintended bias and toxicity in generation [53], the re-emergence of previously unlearned problematic behaviors [55], and overall reductions in model trustworthiness [34]. An alternative direction is to adopt lossless compression to reduce memory footprint without altering model accuracy or behavior.

Prior work has leveraged the sparsity patterns in model parameters to apply lossless compression algorithms for Convolutional Neural Networks (CNNs) [39], [24], [25]. For instance, Eyeriss [24], [25] incorporates run-length coding, while Lane-Compression [39] applies algorithms such as LZW [51] and Deflate [28] to grouped bits to compress CNN parameters. However, the efficacy of these proposals has been demonstrated using dedicated reconfigurable hardware, and therefore they can be impractical for typical CPUs, which are inefficient at fast bit-level operations and decompression.

*Hyungyo Kim and Qirong Xia contributed equally to this work

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

To tackle the challenge in providing effective and efficient lossless compression of model parameters for CPU-based inference, this work exploits the latest on-chip accelerators in commodity CPUs and proposes **LILO**: an **L**LM **I**nference framework supporting **Lo**ssless compression. Specifically, LILO accelerates inference under memory capacity constraints by storing model parameters in a compressed form, significantly reducing the amount of data offloaded to the storage device. It then decompresses them on-the-fly at high throughput using CPU on-chip accelerators. This work makes the following three contributions.

Demonstrating the potential of compressed LLM inference and the limits of relying on CPU cores for decompression. We first analyze the effectiveness of various compression algorithms on BF16 model parameters from Llama3-405B [32] and DeepSeek-R1 [26], both exceeding typical CPU memory capacity. By combining carefully selected compression algorithms with data preprocessing (*i.e.,* byte-grouping), the size of the model parameters of these models can be reduced as low as 67% of their original size. At such compression ratio, storing model parameters in a compressed format and decompressing them on demand can significantly reduce storage access, which, for example, can reduce inference latency by 5.0× in theory for a system running Llama3-405B with 512 GB of memory capacity assuming negligible decompression overhead. However, when decompression is performed on CPU cores, end-to-end latency is reduced only up to 1.2× compared to the uncompressed baseline. This is because the benefits from reduced storage-offloading are offset by the decompression overhead as general-purpose CPU cores struggle to perform high-speed decompression and fine-grained byte-level data manipulation required by the algorithms.

Development of a high-throughput decompression solution leveraging emerging CPU on-chip accelerators. From the set of evaluated algorithms, LILO adopts byte-grouped Deflate for its strong compression ratio and high compatibility with Intel Advanced Vector Extensions (AVX) and In-memory Analytics Accelerator (IAA) accelerators. LILO leverages IAA to accelerate the decompression of the Deflate algorithm while exploiting AVX to efficiently perform the byte-level data reconstruction that reverses the byte-grouping applied during compression preprocessing. To maximize throughput across both accelerators, LILO implements a tightly integrated pipeline with lock-free inter-thread communication, a custom thread pool, and an optimally chosen IAA tuning parameter, *i.e.,* chunk size. With this optimized decompression solution, LILO achieves up to 154 GB/s of decompression throughput, 9.3× higher than the CPU-core-based decompression.

Compressed LLM inference acceleration. With the high-throughput decompression solution integrated with Intel's LLM inference framework, LILO overlaps the inference compute running on Advanced Matrix Extensions (AMX) with decompression running on AVX and IAA by allocating dedicated CPU cores to each task. This enables LILO to minimize resource contention and execution thrashing between compute and decompression, further reducing the inference latency. Furthermore, LILO introduces selective compression, a Mixture-of-Experts (MoE)-aware optimization that minimizes decompression overhead by exploiting the structural asymmetry in MoE models between frequently accessed components (*e.g.,* attention layers and shared experts) and infrequently accessed ones (*e.g.,* routed experts). Rather than compressing the entire model, LILO applies compression only to the infrequently accessed components, which, for example, constitute 97% of the model's total parameter size but can account for as little as 54% of the active parameters during the inference of DeepSeek-R1. Such selective compression reduces the decompression overhead by up to 1.9×, while maintaining a compression ratio nearly identical to that of full-model compression. Combined, LILO achieves up to 4.9× and 4.3× reductions in per-token inference latency for Llama3-405B and DeepSeek-R1, respectively, compared to Accelerate, the uncompressed baseline solely relying on storage-offloading. Figure 1 illustrates the significantly reduced storage-offloading overhead achieved by LILO, while minimizing decompression overhead by leveraging on-chip accelerators.

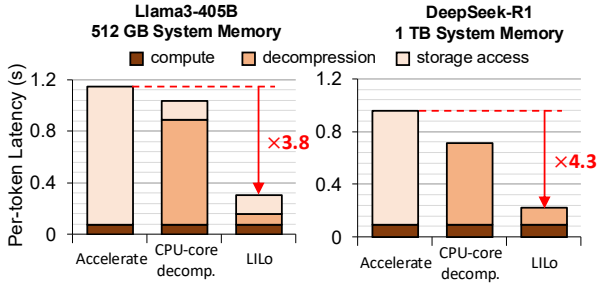


Fig. 1. Per-token latency (normalized by batch size) and its breakdown for HuggingFace Accelerate, compressed LLM inference using CPU cores for decompression, and LILO on input/output token length of 128/256 for a batch size of 64.

II. Background

A. CPU-based Inference

CPU-based inference has gained traction as a complementary and cost-efficient alternative to GPU-centric LLM deployment. Beyond enabling LLM serving for users with limited access to multi-GPU systems, hyperscalers can also leverage underutilized CPU servers to meet a temporarily surging demand for LLM inference services [21], [46], although it is not designed to replace GPU-based inference targeting stringent service level objectives (SLOs). CPU-based inference has become more compelling, especially with servers equipped with the latest Intel CPUs that feature AMX. It significantly enhances matrix-multiplication throughput and, combined with the CPUs' much larger memory capacity than that of GPUs, has demonstrated up to 20 tokens/s for models as large as OPT-175B [38]. Such performance is notable given the community's strong interest in low-cost solutions (*e.g.,* single-GPU inference with data-offloading [49], [50]) with much lower token generation rates (1–2 token/s for OPT-175B).

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

Nonetheless, even CPUs face memory capacity challenges as typical servers deployed by hyperscalers can be equipped with the most cost-effective, mid-capacity DIMMs (*e.g.,* 32 GB or 64 GB), totaling 512 GB–1 TB of memory when all the memory channels are fully populated with such DIMMs. This still falls short of accommodating extremely large models such as Llama3-405B or DeepSeek-R1, which require 768 GB and 1.3 TB of memory, respectively—models of broad interest for their state-of-the-art capability. Note that a high-capacity DIMM (*e.g.,* 256 GB) incurs a superlinearly higher per-GB cost than a mid-capacity DIMM (*e.g.,* 32 GB or 64 GB) [9]. Besides, it requires servers with more expensive power-supply and cooling solutions due to significantly higher power and heat dissipation [47]. Alternatively, CXL memory can be used to increase the memory capacity of servers, but provides only limited memory bandwidth, hurting the memory-bandwidth-sensitive performance of LLM inference. In such scenarios, enabling efficient LLM inference under limited memory capacity becomes crucial, which our work primarily focuses on.

B. Data-offloading during Inference

DeepSpeed Zero-Inference [19] and HuggingFace Accelerate [33] are two prominent frameworks that enable CPU-based inference for LLMs that exceed the CPU memory capacity by offloading the model parameters to storage devices. DeepSpeed Zero-Inference implements a full-model-offloading strategy, whereas HuggingFace Accelerate supports both full- and partial-model-offloading to optimize the performance. However, storage-offloading introduces significant performance overhead due to the limited bandwidth of storage devices. Figure 2 illustrates the end-to-end inference latency of Llama3-405B under varying percentages of the parameters offloaded to NVMe SSD storage, characterized using HuggingFace Accelerate. The performance analysis was conducted on a 6$^{\text{th}}$-generation Intel Xeon Scalable Processor (codenamed Granite Rapids, or GNR) with 128 cores for input/output token lengths of 256/32. For a batch size of 1, latency increases by 8× even when just 10% of the parameters are offloaded to storage, and reaches 35× when 50% of parameters are offloaded. While increasing the batch size can amortize this overhead by performing more compute once the parameters are fetched from the storage, the penalty remains substantial. For instance, even at a large batch size of 64 with 50% of the parameters offloaded, the latency is still 7× higher than the baseline, underscoring the critical performance bottleneck introduced by storage offloading.

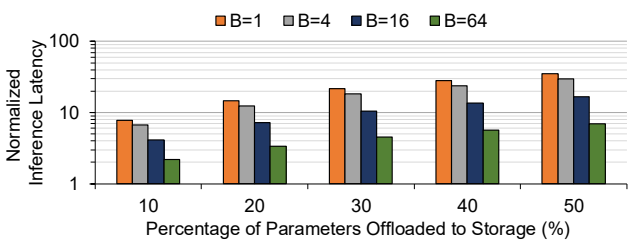


Fig. 2. Inference latency of Llama3-405B with varying proportions of model parameters offloaded to SSD, normalized to the case without offloading. Measured on Intel GNR CPUs with NVMe SSDs, using HuggingFace Accelerate for storage offloading across different batch sizes (B).

C. Lossless Compression Algorithms

Popular lossless compression algorithms roughly fall into three categories: entropy coding, dictionary-based compression, and run-length encoding. Entropy coding, such as Huffman coding [35] and arithmetic coding [1], assigns shorter codes to more frequent symbols. Dictionary-based methods, including LZ77 [56], LZ78 [57], and LZ4 [42] replace recurring substrings with references to earlier occurrences, reducing redundancy. Lastly, run-length encoding [48] compresses sequences of repeated values by storing the value once alongside its repetition count, making it particularly effective for data with long runs of identical symbols. Deflate [28] is one of the most widely used lossless compression algorithms, which combines dictionary-based method (LZ77) and entropy coding (Huffman coding) to effectively balance compression ratio and speed. Numerous hardware accelerators for lossless compression implement Deflate, including Intel IAA [6], IBM Power9 on-chip accelerator [18], and NVIDIA BlueField DPUs [22].

D. Intel CPU On-chip Accelerators

Figure 3 illustrates the overall architecture of Intel Xeon Scalable Processors from the 4$^{\text{th}}$ generation onward and the details of three accelerators, IAA, AMX, and AVX.

IAA is an on-chip accelerator dedicated for computationally intensive tasks, including compression/decompression specifically focusing on the Deflate algorithm, encryption, and analytics. The bottom left box in Figure 3 illustrates the architecture of IAA. An IAA instance consists of eight work queues and eight processing engines. IAA transfers and processes data in the granularity of a data chunk, referred to as a job, the size of which can range from 1 KB to 2 MB. Each job is represented by a job descriptor, which specifies the operation, Huffman coding type, pointers to the source and destination data chunks, and their corresponding sizes. Processing engines then fetch descriptors from the work queue, and through arbiter-managed scheduling, forward them to processing pipes for analytics, compression, or decompression. Lastly, Query Processing Library (QPL) [7] provides an interface between user applications and accelerator hardware.

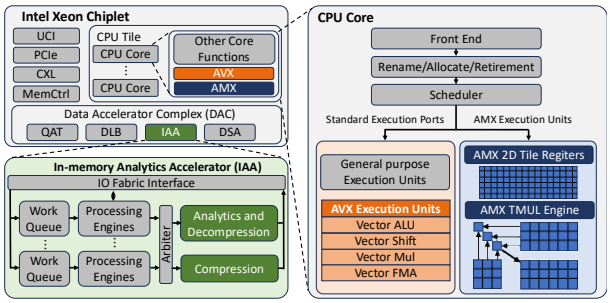


Fig. 3. Intel Xeon Scalable Processors from the 4$^{\text{th}}$ generation onward, with detailed illustrations on AMX, AVX, and IAA.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

AMX is a matrix-multiplication accelerator, along with ISA support, starting from the 4th-generation Xeon CPUs (SPR). The scheduler dispatches AMX instructions, such as tile load/store and accelerator commands, to dedicated AMX execution unit (Figure 3). AMX execution unit consists of two components: (1) a 2D array of register tiles and (2) Tile matrix multiply unit (TMUL) [4], which are designed to support INT8 and BF16 formats. The tiles store sub-arrays of matrices; the TMUL, a 2D array of multiply-add units, operates on these tiles. TMUL's 2D tile-based matrix execution enables higher ops/cycle by exploiting greater parallelism than 1D compute engines such as AVX engines.

AVX is a SIMD instruction set to accelerate data-parallel computations by enabling the processing of multiple data elements in parallel. AVX operates on 256-bit (AVX, AVX2) and 512-bit (AVX-512) wide YMM and ZMM registers, allowing simultaneous computation on multiple integers or floating-point numbers. In modern CPU architectures, AVX instructions are executed by vector execution units that share the standard execution ports with general-purpose scalar and floating-point units. These shared ports enable parallel execution of AVX instructions alongside scalar operations.

III. Compressed LLM Inference: The Potential and Limitations

To mitigate the performance overhead of storage-offloading discussed in §II-B, we propose *compressed LLM inference*. During compressed LLM inference, model parameters are stored in a losslessly compressed format and decompressed on-the-fly during inference. By reducing the volume of model parameters, compressed LLM inference can significantly reduce the amount of data offloaded to storage when the model size exceeds the CPU memory capacity, resulting in accelerated inference. We first analyze the compressibility of LLM parameters of Llama3-405B and DeepSeek-R1 across different compression algorithms. We then evaluate the decompression throughput of these algorithms on CPU cores. Finally, we discuss the impact of decompression throughput on the latency reduction achievable via compressed LLM inference.

A. Compressibility of BF16 Model Parameters

BF16 data format. We focus on model parameters stored in BF16 format. BF16 encodes values using 1 sign bit, 8 exponent bits, and 7 mantissa bits, offering the same dynamic range as FP32 but at half the bit-width. Due to this balance between precision and efficiency, BF16 has become the default parameter format in many state-of-the-art models, as reflected in the HuggingFace text generation model catalog [2]. Although FP8 models like DeepSeek-R1 have emerged recently, even their parameters are losslessly converted to BF16 offline for CPU deployment, as FP8 is not supported on CPUs.[1]

Probability distribution of parameters. The upper left plot in Figure 4 illustrates the distribution of the parameters of Llama-3-405B in the BF16 data type. As the parameter values cluster tightly around a narrow range, exponent value of the parameters result in extremely concentrated distribution, opposed to the relatively evenly distributed sign and mantissa values of the parameters as illustrated in Figure 4. The entropy of each part is calculated at 1 bit, 1.83 bits, and 6.97 bits, respectively, hinting that the exponent part can benefit significantly from compression algorithms. Similarly, the distribution of the 1st-byte, consisting of 1 sign bit and 7 most significant bits of the exponent, also shows a similar concentrated pattern with 2.83 bits in entropy. The 2nd-byte draws a distribution close to the uniform distribution with 7.97 bits in entropy. In the rest of the paper, we refer to the 1st-byte and 2nd-byte of BF16 as the Upper Byte (UB) and Lower Byte (LB).

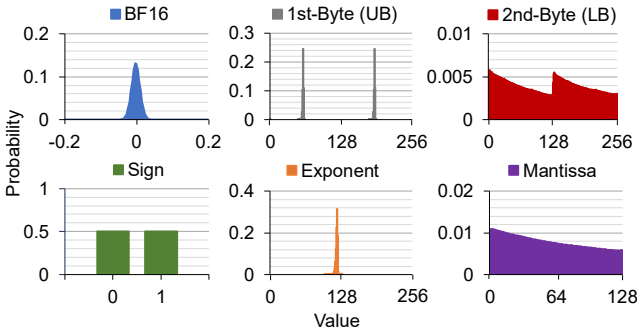


Fig. 4. The probability distribution of BF16 parameter values, 1st/2nd byte of the parameters (Upper Byte (UB)/Lower Byte (LB)), and sign, exponent, and mantissa of the parameters of BF16 Llama3-405B model.

Compression ratio. Table I reports the compression ratio, defined as (compressed size)/(original size), of Llama3-405B and DeepSeek-R1. We evaluate two compression algorithms: LZ4 [42], a lightweight run-length-based algorithm similar to that used in Eyeriss [24], [25], and Deflate [28], which offers more compression at the expense of slower decompression. To exploit the entropy difference between byte positions of LLM parameters, we adopt byte-grouping, a variant of lane-grouping [39], where UB and LB of parameters are grouped separately and compressed independently. This isolates the low-entropy exponent/sign bytes from the high-entropy mantissa bytes, avoiding entropy interleaving that would otherwise hinder match lengths and flatten symbol distributions. While LZ4 without byte-grouping fails to compress the model, byte-grouping reduces its compression ratio to 87%. Deflate, in contrast, achieves 79% and 72% even without byte-grouping, and improves further to 71% and 67% with byte-grouping for Llama3-405B and DeepSeek-R1, respectively. We also find that the LB group is incompressible with either LZ4 or Deflate.

> **Insight-1:** Lossless compression algorithms can reduce the model parameter size by up to 33% without compromising the model accuracy and behavior.

B. Impact of Decompression Speed

Decompression throughput. Table I also reports the decompression throughput of LZ4 and Deflate, with and without byte-grouping, measured on a 128-core Intel GNR CPU.

[1]The conversion process using DeepSeek's official code [27] takes 30–40 minutes, making on-the-fly conversion during inference infeasible.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPRESSION RATIO (CR, %) AND DECOMPRESSION THROUGHPUT (DT, GB/S) OF BF16 LLM PARAMETERS FOR DIFFERENT COMPRESSION ALGORITHMS WITH AND WITHOUT PREPROCESSING. DECOMPRESSION IS PERFORMED ON A 128-CORE GRANITE RAPIDS.

Preprocessing	Compression Algorithm	Llama3-405B		DeepSeek-R1	
		CR	DT	CR	DT
None	LZ4	100.4	149.8	100.4	194.2
	Deflate	79.3	10.5	72.2	9.6
Byte-grouping	LZ4	87.3	55.1	87.3	64.1
	Deflate	70.9	16.5	67.5	9.3

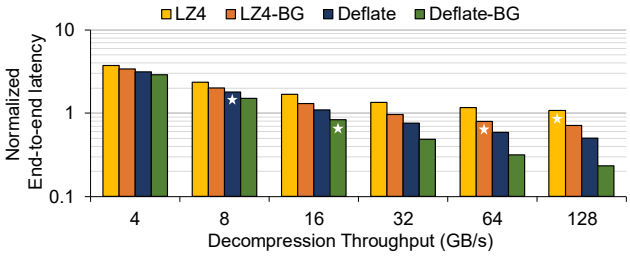


Fig. 5. Potential latency of compressed Llama3-405B inference under 512 GB memory for various algorithms, normalized to the uncompressed baseline. BG denotes byte-grouping; x-axis shows assumed decompression throughput; starred bars indicate CPU-based decompression cases.

For cases with byte-grouping, the reported decompression throughput includes the cost of postprocessing, where the byte-grouped parameters are reassembled into the original BF16 format, a step referred to as BF16-reconstruction hereafter. The model parameters are concatenated and then partitioned into 128 equal-sized chunks, each decompressed in parallel across 128 cores to maximize throughput using Python `zlib` [17] `lz4` [11] libraries for Deflate and LZ4, respectively.

LZ4 without byte-grouping achieves impressively high decompression throughputs of 149.8 GB/s and 194.2 GB/s for Llama3-405B and DeepSeek-R1, respectively. However, such high throughputs are largely attributed to the near-zero compression ratios, effectively bypassing any meaningful decompression. With byte-grouping, the throughput of LZ4 drops to 55.1 GB/s and 64.1 GB/s for Llama3-405B and DeepSeek-R1, respectively, due to the additional overhead of BF16 reconstruction and increased decompression workload. Deflate exhibits lower decompression throughput overall, ranging from 9.3–16.5 GB/s depending on the model and whether byte-grouping is applied. This is because Deflate is a more complex compression algorithm than LZ4, involving multiple steps such as Huffman decoding and LZ77 back-referencing, thereby trading decompression speed for higher compression ratios. Note that the decompression performance is data-dependent, varying across models and whether byte-grouping is applied.

Compressed LLM inference latency. Figure 5 projects the potential inference latency for a compressed Llama3-405B model operating under a 512 GB memory constraint, normalized to an uncompressed baseline. Our analysis evaluates various compression algorithms across a range of decompression throughputs, using an input/output of 256/32 tokens and a batch size of 1. The projected latency is obtained by summing the on-the-fly decompression and storage-offloading overheads with the baseline inference latency. The decompression cost is calculated from the assumed throughput, while the storage-offloading overhead is determined by the amount of data offloaded to meet the memory capacity limit. This offloaded amount depends on the total memory footprint (compressed parameters, KV cache, and activations).

The corresponding latency overhead is then characterized using HuggingFace Accelerate by offloading the same amount of data. For each algorithm, the decompression throughput achieved by CPU cores given in Table I is indicated by a starred bar, with values rounded to the nearest assumed level. Only modest latency reduction of up to 1.2× are observed for Deflate and LZ4 with byte-grouping, while the other methods rather increase latency. This is primarily due to limited decompression throughput, which offsets the benefits of reduced storage offloading. However, substantial latency reductions become possible as decompression throughput increases. For example, using Deflate with byte-grouping achieves 5.0× improvement when decompressed at 128 GB/s. These findings underscore the importance of high-throughput decompression to ensure that the benefits of reduced storage access are not offset by decompression overhead.

Insight-2: While lossless compression has the potential to significantly reduce inference latency under memory constraints, its benefits are constrained by the limited decompression throughput of CPU cores.

C. Opportunities and Challenges with On-chip Accelerators

Intel's on-chip accelerators provide opportunities for accelerating the decompression of Deflate with byte-grouping, which shows the highest potential to benefit from compressed LLM inference. IAA offers hardware-accelerated decompression for Deflate, while AVX enables wide vectorized operations ideal for BF16 reconstruction. However, without careful tuning of accelerator operating parameters, efficient thread management, and coordinated orchestration, substantial performance variation and resource under-utilization can occur. For instance, differences in thread management implementations and accelerator operating parameters can cause up to 2× variation in decompression throughput (§IV-C). Moreover, lacking proper coordination between these accelerators can lead to significant idle time, yielding up to 1.9× lower performance without fine-grained pipelining and 1.6× lower performance without overlapping decompression and inference computation (§V-C). Therefore, fully exploiting these accelerators requires careful orchestration, thread management, and operating-parameter tuning to maximize decompression and compute efficiency, forming the motivation of our work.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

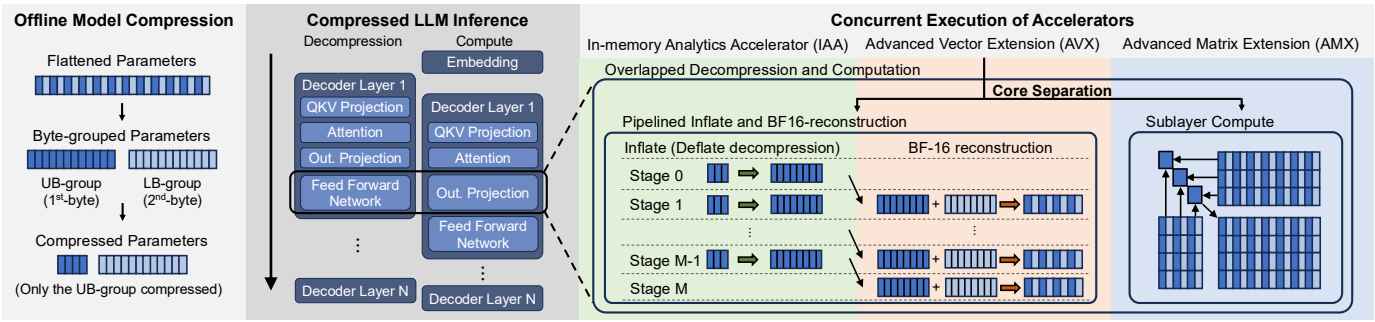


Fig. 6. Overview of LILO.

IV. LILO: Accelerating Compressed LLM Inference with CPU On-chip Accelerators

To realize the potential of compressed LLM inference, we present LILO, a framework that achieves high-throughput decompression by leveraging Intel's on-chip accelerators. LILO efficiently orchestrates pipelining and overlapping across IAA, AVX, and AMX—the latter used for accelerating LLM computation. LILO implements a compression method that combines the Deflate algorithm with byte-grouping, selected for its superior compression ratio and its high compatibility with hardware accelerators' capabilities. In the following subsections, we first present the overview of LILO. We then describe the implementation of its high-throughput decompression solution, which leverages IAA and AVX. Finally, we describe compressed LLM inference acceleration optimizations including compute/decompression overlap and selective compression.

A. LILO *Overview*

LILO consists of two stages: (1) offline model parameter compression, and (2) compressed LLM inference with on-the-fly decompression, as illustrated in Figure 6. In the offline stage, LILO iterates through the Decoder layers and applies byte-grouping to each sublayer's parameters, separating each BF16 parameter into `UB-group` and `LB-group`. As discussed in §III-A, `UB-group` carries most of the compressibility, while `LB-group` remains incompressible. Therefore, only `UB-group` is compressed, and `LB-group` is stored uncompressed. During inference, the compressed model parameters are decompressed by reversing the offline compression. Specifically, `UB-group` undergoes Inflate, the decompression process of the Deflate algorithm, and is then combined with the uncompressed `LB-group` for BF16-reconstruction. We leverage IAA and AVX to accelerate Inflate and BF16-reconstruction, respectively, and implement a decompression pipeline to maximize throughput. Table II summarizes LILO's decompression throughput, which is 9.3–14.8× higher than that of CPU cores.[2] During the pipelined Inflate and BF16-reconstruction for Llama3-405B parameters, the IAA decompression engines sustain an occupancy of 85% with a queue depth of 13 on average. Furthermore, decompression is overlapped with inference computation at the sublayer granularity by dedicating separate sets of cores to each task, thereby minimizing resource contention and execution thrashing between decompression and compute.

[2]We further validated that parameters decompressed from UB-only and UB+LB compression exactly match the originals bit-for-bit, and confirmed the identical perplexity on the OpenOrca validation set between LILO and the uncompressed baseline. As LILO applies *lossless* compression solely to model parameters, such validation ensures complete preservation of inference accuracy.

TABLE II
INFLATE, BF16-RECONSTRUCTION, AND THE COMBINED DECOMPRESSION THROUGHPUT (GB/S) OF CPU CORE BASELINE AND LILO FOR LLAMA3-405B AND DEEPSEEK-R1 PARAMETERS.

Function	Llama3-405B		DeepSeek-R1	
	CPU core	LILO	CPU core	LILO
Inflate	11.27	82.54	5.11	72.79
BF16-reconstruction	111.36	194.04	103.68	191.34
Decompression (Inflate + BF16-reconstruction)	16.49	153.72	9.31	136.82

B. Hardware Acceleration

IAA-accelerated Inflate. Inflate is executed on IAA in three stages: ① allocation and initialization of job descriptors, ② descriptor submission to initiate Inflate, and ③ deallocation and clean up of descriptors and associated metadata. As stages ① and ③ incur overhead for different runs, we separate these two stages into a separate module and run once at start-up of the inference, as the descriptors can be reused and dynamically adjusted across runs. Only stage ②, the actual Inflate step, is called by the IAA Inflate module during the inference. Descriptors are released upon completion of inference.

IAA Inflate performance depends primarily on the chunk size that IAA operates on. For the GNR CPU used in our setup (detailed in §V), a socket contains 4 IAA accelerators with 8 decompression engines each, totaling 32 engines. To maximize the utilization of the 32 engines for parallel execution, it is beneficial to submit 32 job descriptors simultaneously. To fully utilize the hardware, chunk sizes should be small enough such that each parameter set is divided into at least 32 chunks, fully exploiting the parallelism of the 32 engines. However, small chunk sizes incur high CPU-IAA communication overhead,

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

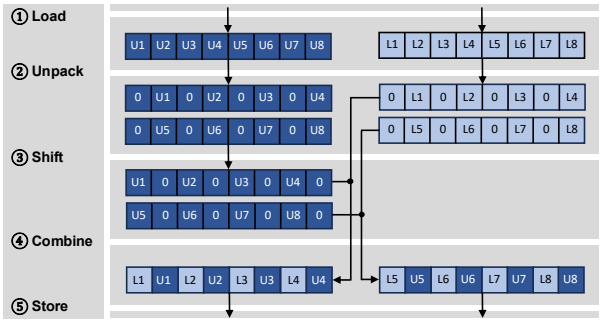
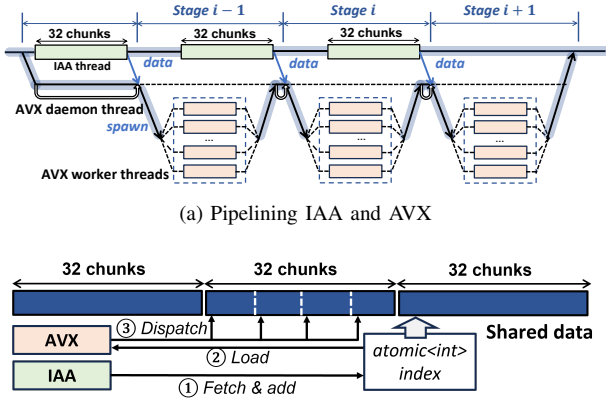


Fig. 7. AVX512-accelerated BF16-reconstruction process.



(a) Pipelining IAA and AVX

(b) Data sharing method

Fig. 8. (a) Timing diagram of pipelining IAA decompression and AVX BF16-reconstruction. (b) Data sharing and inter-thread communication method.

decreasing the throughput. By increasing the chunk size, the communication overhead can be amortized more effectively, but may result in engine under-utilization. Therefore, chunk size must be carefully tuned to balance engine utilization and communication efficiency.

AVX512-accelerated BF16-reconstruction. We leverage AVX512 intrinsics to accelerate BF16-reconstruction, focusing on their ability to load and store wide vectors and perform byte-level data operations efficiently within and between AVX512 registers. Figure 7 illustrates the AVX512-accelerated BF16 reconstruction process, consisting of five stages. ① Load: first, two 64×8-bit elements from each of `UB-group` and `LB-group` are loaded into 512-bit registers using the `_mm512_loadu_si512` instruction. ② Unpack: the lower and upper 256-bit halves are extracted via `_mm512_castsi512_si256` and `_mm512_extracti64x4_epi64`, then zero-extended to 16-bit integers using `_mm512_cvtepu8_epi16`. ③ Shift: the 8-bit UBs are left-shifted with `_mm512_slli_epi16` to prepare for bitwise merging. ④ Combine: the shifted UBs are combined with the LBs using `_mm512_or_si512`, a bitwise OR operation to produce the final 16-bit BF16 values. ⑤ Store: finally, the reconstructed BF16 data is written back to memory in two 512-bit chunks using `_mm512_storeu_si512`. To exploit both data- and core-level parallelism, we invoke AVX512 instructions across multiple threads pinned to separate CPU cores. Thread binding via CPU affinity minimizes preemption and context switches while preserving cache locality, further improving throughput.

C. Pipelined Decompression Module

Pipeline stages. A basic implementation of decompression executes IAA Inflate followed by the AVX BF16-reconstruction sequentially. However, it yields suboptimal performance due to two reasons: (1) the AVX BF16-reconstruction must wait for the entire `UB-group` to be decompressed, despite earlier decompressed data chunks becoming available sooner, and (2) IAA job submission and synchronization is handled using only one core, leaving most of the CPU cores idle during Inflate.

To address the underutilization of computation resources, we pipeline IAA Inflate and AVX BF16-reconstruction to maximize decompression throughput. Since IAA Inflate operates at the granularity of data chunks, we define each pipeline stage by the number of chunks processed concurrently. As discussed in §IV-B, submitting 32 data chunks in parallel fully utilizes IAA's decompression engines. Therefore, each pipeline stage comprises 32 chunks, as illustrated in Figure 8a. At the i^{th} pipeline stage, multiple worker threads use AVX instructions to perform BF16-reconstruction on the 32 Inflated chunks produced by IAA from the previous $(i-1)^{th}$ stage.

Lock-free inter-thread communication. To eliminate lock-induced contention between the IAA thread and the AVX worker threads, we develop a lock-free synchronization mechanism using an atomic integer, `atomic_idx`. The design leverages the fact that AVX worker threads only need to know which portions of the shared data array have been inflated. As depicted in Figure 8b, IAA thread first atomically `fetch & add` the `atomic_idx`, signaling that all data before that index is ready for BF16-reconstruction. A dedicated daemon thread continuously polls `atomic_idx` for updates. Upon detecting an update of `atomic_idx`, the daemon thread dispatches worker threads to process the newly available data in continuous chunks. This lock-free coordination removes synchronization overhead across pipeline stages, ensuring that Inflate the BF16-reconstruction proceeds without blocking.

Optimizing thread management. A straightforward way to implement pipelined decompression is to use OpenMP [13], which simplifies multi-threading programming (referred to as `Decomp-omp`). However, `Decomp-omp` suffers from the following drawbacks. First, while a daemon thread coordinates with the IAA thread and spawns AVX worker threads, it primarily spins on the atomic variable without performing useful work. Second, OpenMP spawns worker threads for AVX BF16-reconstruction from the daemon thread at runtime, introducing overhead from task division among worker threads and runtime scheduling across processors. Moreover, OpenMP's programming interface limits flexibility to tailor thread management overhead for our workload.

To overcome these limitations, we develop a specialized thread-pool design tailored to our pipeline (referred to as

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

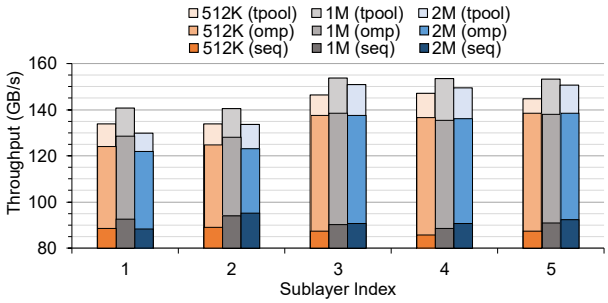


Fig. 9. Decompression Throughput (GB/s) of different implementations, with varying IAA chunk sizes, on 5 different sublayers in Llama3-405B.

`Decomp-tpool`). The thread pool allocates one thread for IAA decompression and multiple worker threads for AVX BF16-reconstruction during initialization. These threads remain idle when not in use, waking up only when a task is initiated and enqueued, and returning to sleep upon completion. We use semaphores solely for task initiation and termination, while all runtime coordination is handled through lock-free atomic operations, as described earlier.

Figure 9 presents the decompression throughput of different decompression implementations with varying chunk sizes, evaluated on the model parameters from distinct sublayers of Llama3-405B. Our thread-pool-based implementation, `Decomp-tpool`, achieves up to 1.1–1.2× and 1.4–1.7× higher throughput compared to `Decomp-omp` and `Decomp-seq`, respectively. Among the different chunk sizes, 1 MB case yields the highest throughput by effectively balancing decompression engine utilization and CPU–IAA communication overhead.

D. Compute/Decompression Overlap

We integrate our decompression implementation to Intel Extensions for PyTorch (IPEX) library, which leverages the latest AMX technology to accelerate LLM inference on Intel CPUs. During inference, decompression can be performed just-in-time before the parameters are used for computation. However, such sequentially interleaved execution can disrupt cache locality, preventing subsequent sublayers from reusing the outputs of earlier sublayers that reside in cache. This results in increased memory traffic, leading to inference slowdown. To mitigate this, LILO assigns decompression and decoder computation to separate sets of physical CPU cores: one set dedicated to decompression, and the other to computation. Then, LILO overlaps the decompression of the $(i+1)^{th}$ sublayer with the computation of i^{th} sublayer, as illustrated in Figure 10. This approach ensures sublayer outputs to remain in core-local caches for immediate reuse. While Hyper-Threading can be considered another option to implement overlapping, the microarchitectural contention between AMX and AVX units often leads to performance degradation [3]. We adopt sublayer-level overlapping instead of Decoder-layer granularity for two reasons. First, only the uncompressed weights of the sublayers within the Decoder layer must be buffered, compared to buffering two decoder layers, resulting in a 2× reduction in buffer size. Second, we observe that decompressing all the Decoder-layer weights at once increases contention in the shared last-level cache between the compute and decompression streams, achieving smaller improvement compared to sublayer-granularity overlapping.

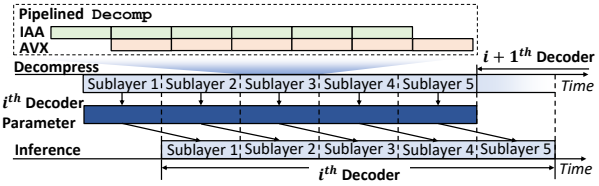


Fig. 10. Timing diagram of overlapped decompression and inference computation in sublayer granularity.

E. Selective Compression: Optimization to MoE Models

MoE architecture has been increasingly adopted in state-of-the-art LLMs, including DeepSeek-R1 [26], Llama 4 [15], Mixtral [36], and GPT-4 [44]. MoE architecture differs from conventional dense models in a way that only a subset of parameters is accessed frequently, despite representing a minor fraction of the total model parameters. Exploiting such a characteristic, we propose selective compression to minimize the decompression overhead during inference.

Selective compression. MoE model parameters can be grouped into two types: shared parameters, which are used by all inputs (such as those in dense decoder layers, shared experts, and attention modules), and routed parameters, which are conditionally activated depending on the input token (such as routed experts). Table III shows the breakdown of activated and total parameters in DeepSeek-R1 during the decoding stage, categorized into shared and routed parameters. The average number of unique experts is estimated by modeling the expert selection as a union of random subsets under a uniform distribution [20]. Although routed parameters make up more than 97% of the model's total parameters, they only contribute as low as 54% of the activated parameters during decoding when the batch size is 1. To take advantage of this imbalance, LILO selectively compresses only the routed parameters, leaving shared parameters uncompressed. This strategy reduces the

TABLE III
BREAKDOWN OF ACTIVATED PARAMETER DURING THE DECODING STAGE AND THE TOTAL PARAMETER OF DEEPSEEK-R1 INTO SHARED AND ROUTED PARAMETERS ACROSS VARYING BATCH SIZES.

Batch Size	Parameter Size (Percentage)			
	Activated Parameters		Total Parameters	
	Shared	Routed	Shared	Routed
1	32 GB (46%)	38 GB (54%)	32 GB (3%)	1.2 TB (97%)
4	32 GB (18%)	143 GB (82%)		
16	32 GB (6%)	485 GB (94%)		
64	32 GB (3%)	1.1 TB (97%)		

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

decompression workload by up to 1.9× since only the routed parameters need to be decompressed during inference while incurring a slight increase in compression ratio, from 67.5% to 68.3%, compared to compressing all parameters.

While we choose to selectively compress all the routed experts in this work, it can be further extended to dynamic selective compression, which profiles frequently routed ("hot") experts at runtime and leaves them uncompressed as well. Since expert activation patterns are often skewed and input-dependent—*e.g.,* fewer than 6% of experts account for over 64% of activations in Mixtral and Llama 4 [31], [45]—such a dynamic strategy can further reduce the decompression overhead under non-stationary workloads.

V. Evaluation

A. Experimental Setup

System setup and methodology. We evaluate LILO on a server equipped with a 128-core Intel 6th-Generation Xeon Scalable Processor (Granite Rapids, GNR), as summarized in Table IV. The inference throughput of LILO is compared against an uncompressed baseline for Llama3-405B and DeepSeek-R1 across varying memory capacity constraints. LILO is implemented on top of Intel Extension for PyTorch (IPEX) [5], which provides AMX-optimized inference kernels for LLM inference on Intel CPUs. The execution path can be configured via a runtime flag to select either LILO or fall back to the default IPEX inference, depending on the available host DDR memory capacity. However, existing storage-offloading implementations, such as HuggingFace Accelerate [33] and DeepSpeed Zero-Inference [19], are currently incompatible with IPEX's optimized inference. To circumvent this incompatibility, we model the storage-offloading overhead separately to project the inference latency under various memory constraints. First, we construct reduced 1/3-scale variants of Llama3-405B and DeepSeek-R1 that fit entirely within our evaluation system DDR memory during inference. The reduced Llama3-405B comprises the first 42 decoder layers of the original model, while the reduced DeepSeek-R1 includes one dense decoder layer followed by 19 MoE decoder layers. For both LILO and the baseline, we measure inference latency using the reduced models without storage offloading, then scale the results by 3×. To this scaled base latency, we add a separately modeled storage-offloading overhead. The required storage-offload data size is calculated based on the total memory footprint (parameters, KV cache, and activations) and the system's assumed memory capacity. We then derive the corresponding storage-offloading overhead from a performance curve we characterized using HuggingFace Accelerate, which maps the volume of offloaded data to its resulting latency overhead. We leave direct integration of offloading support into IPEX as future engineering work. For the core allocation ratio and chunk size configuration in LILO, we allocate 64, 63, and 1 cores to AMX compute, AVX BF16 reconstruction, and the IAA daemon thread, respectively, and set the IAA chunk size to 1 MB, which we verify in §V-D to deliver the best performance.

TABLE IV
Evaluation system.

GNR system	Description
CPU	Intel® Xeon® 6980P CPU@2.0GHz, 128 cores and 504 MB LLC per CPU
Memory	12 × DDR5-6400 channels, 768 GB
Storage	Micron 7450 NVMe M.2 SSD, PCIe 4.0 ×4, 480 GB
On-chip IAA	4 on-chip IAAs per CPU, QPL v1.7.0
OS (kernel)	Ubuntu 22.04.5 LTS (Linux kernel 6.8.0-49-generic)

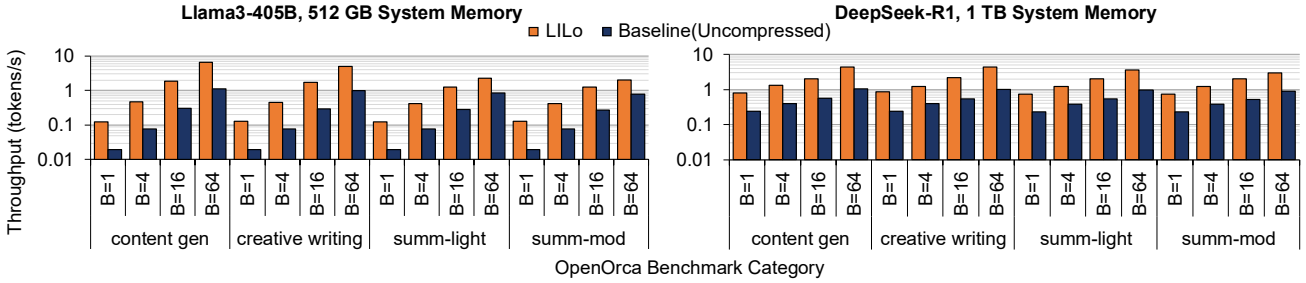
Evaluation points. We evaluate inference throughput using representative input and output token length pairs derived from the OpenOrca dataset [40]. The benchmark includes four task categories: content generation, creative writing, summarization-light, and summarization-moderate with average input/output token lengths of 128/256, 512/512, 1024/128, and 1566/256, respectively. For each category, we construct an input that matches the average input length and set the generation parameters to produce the corresponding average number of output tokens. To prevent our input example from inducing a fixed or biased expert routing pattern in DeepSeek-R1, we override the model's routing decisions with uniformly random expert selection for each token during inference. We evaluate performance across the batch sizes from 1 to 64.

B. Performance Evaluation

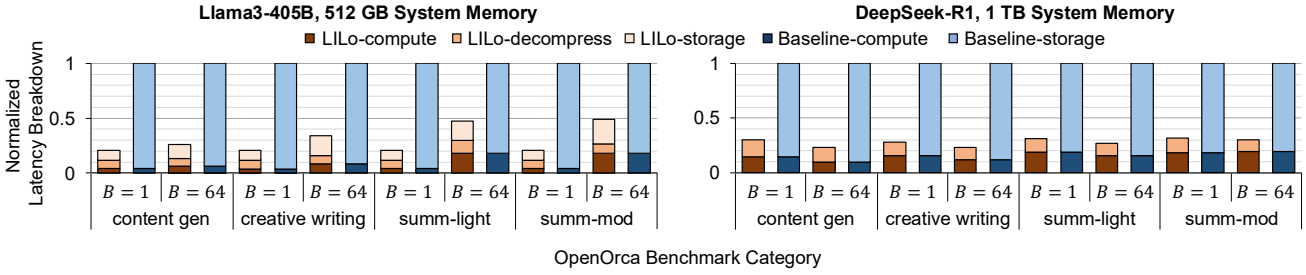
Throughput improvement. Figure 11a presents the inference throughput of Llama3-405B and DeepSeek-R1, comparing LILO with the uncompressed baseline under 512 GB and 1 TB memory capacity constraints, respectively, across benchmark categories and batch sizes. For Llama3-405B, LILO consistently achieves 2.0–4.9× higher throughput than the baseline. The improvement declines with larger batch sizes and longer total sequence length (input+output) as the KV cache size scales with both, forcing LILO to offload more parameters to storage. For example, in summarization–moderate task, the storage-offloaded parameters with LILO increase from 23 GB to 81 GB as batch size increases from 1 to 64, while the baseline increases from 243 GB to 301 GB. Therefore, the relative benefit of reduced storage access decreases, resulting in an overall improvement drop from 4.8× to 2.0×. In contrast, for content generation task, KV cache grows more slowly due to shorter sequence, and LILO's storage-offload only increases from 23 GB to 34 GB as batch size increases from 1 to 64, decreasing the improvement modestly from 4.9× to 3.8×.

For DeepSeek-R1, LILO consistently achieves 3.1–4.3× higher throughput compared to baseline across the benchmark categories and batch sizes. Across all cases, LILO completely avoids storage-offloading by compressing the model size from 1.25 TB to 854 GB and benefiting from the small KV cache sizes with DeepSeek-R1's Multi-head Latent Attention (MLA). As a result, the improvement with LILO maintains consistently high even for benchmark categories with long sequence lengths and large batch sizes, as it continues to operate within DDR capacity without offloading.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.



(a) Inference throughput comparison between LILO and baseline (uncompressed)



(b) Latency breakdown of LILO and baseline

Fig. 11. Inference throughput and latency breakdown comparison between LILO and the uncompressed baseline for Llama3-405B and DeepSeek-R1, under memory capacity constraints of 512 GB and 1 TB, respectively. Measurements are taken using representative input/output lengths from the OpenOrca dataset, with batch size (B) swept from 1 to 64. For the latency breakdown, LILo's latency is normalized to the uncompressed baseline.

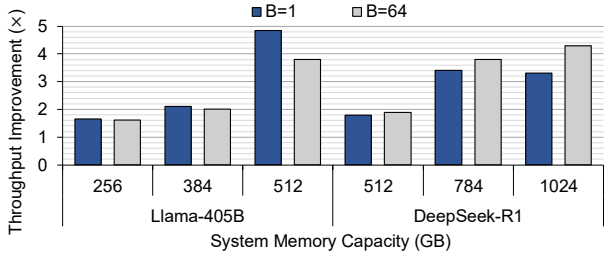


Fig. 12. Llama3-405B and DeepSeek-R1 inference throughput improvement with LILO compared to the uncompressed baseline under varying memory capacity for content generation category and batch sizes (B) of 1 and 64.

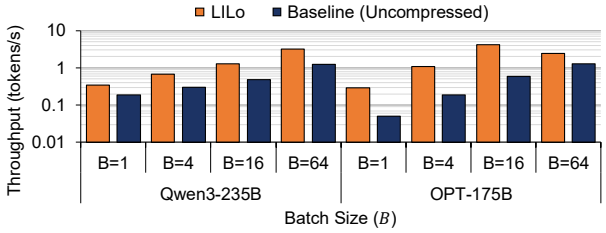


Fig. 13. Inference throughput of LILO and the uncompressed baseline for Qwen3-235B and OPT-175B, under 256 GB system memory capacity for content generation and batch size (B) swept from 1 to 64.

Latency breakdown. Figure 11b illustrates the inference latency breakdown of LILO and the baseline. For Llama3-405B, storage-offloading accounts for 37–53% of the total latency in LILO, while 82–96% for the baseline. Within the remaining 47–64% of LILO's latency, decompression contributes an overhead of 0.5–1.9× relative to compute latency. These results demonstrate that LILO effectively mitigates the storage bottleneck, while maintaining a relatively low decompression overhead through its high 153.7 GB/s decompression throughput. The decompression overhead decreases with larger batch sizes and benchmarks featuring longer input sequences with shorter output sequences (summarization-light and moderate), as decompression latency scales only with the output length, while compute latency scales with total length and batch size.

For DeepSeek-R1, LILO completely avoids storage-offloading, whereas storage-offloading accounts for 80–90% of the baseline's total latency. The decompression overhead of LILO ranges from 69–141% relative to compute latency at batch size 1, lower than the 158–195% overhead for Llama3-405B. This is due to LILo's selective decompression, which only decompresses the routed experts during inference.

Under different memory capacity constraints. Figure 12 demonstrates the throughput improvement of Llama3-405B and DeepSeek-R1 with LILO compared to the baseline across varying system memory capacity for the content generation task. For Llama3-405B, the improvement peaks at 512 GB, reaching 4.9× and 3.9× for batch sizes 1 and 64, respectively. As memory capacity decreases to 256 GB, the improvement drops to 1.7× and 1.6×, respectively, due to a diminishing gap in the amount of model parameters offloaded by LILO and the baseline. Specifically, the offloading ratio between baseline and LILO shrinks from 11.0× to 1.8× at batch size 1, and from 7.5× to 1.7× at batch size 64.

For DeepSeek-R1, LILO achieves peak throughput improvements of 3.3× and 4.3× at 1 TB system memory for batch sizes 1 and 64, respectively, with decompression

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ABLATION STUDY OF LILO'S COMPONENTS. INFERENCE THROUGHPUT (TOKENS/S) OF LLAMA3-405B AND DEEPSEEK-R1 MEASURED FOR CONTENT GENERATION AND BATCH SIZE OF 1.

Ablation Setting	Throughput (tokens/s)	
	Llama3-405B	DeepSeek-R1
All optimizations	0.13	0.81
No overlapping	0.12	0.43
No pipelining	0.10	0.49
No selective compression	–	0.67
No IAA/AVX accelerators	0.02	0.13

TABLE VI
SENSITIVITY STUDY OF LILO'S INFERENCE THROUGHPUT FOR LLAMA3-405B ACROSS DIFFERENT CORE ALLOCATIONS AND CHUNK SIZES FOR CONTENT GENERATION AND BATCH SIZES (B) OF 1 AND 64.

Chunk Size	Core Allocation (AMX, AVX, IAA)	Throughput (tokens/s)	
		$B = 1$	$B = 64$
1 MB	(32, 95, 1)	0.107	4.30
	(64, 63, 1)	**0.133**	**6.73**
	(96, 31, 1)	0.107	5.88
512 KB	(64, 63, 1)	0.125	6.49
1 MB		**0.133**	**6.73**
2 MB		0.128	6.11

throughput reaching 95–135 GB/s. As memory capacity decreases to 768 GB, LILO begins to offload a small portion of parameters (86–94 GB), incurring a moderate increase in latency. However, the uncompressed baseline experiences a larger rise in storage-offload latency as its offloaded data grows more substantially (by 256 GB), resulting in a similar proportional slowdown. Consequently, the relative throughput gain of LILO remains similar. When capacity drops to 512 GB, LILO offloads more data (334–342 GB), and storage offloading becomes a major contributor to total latency. Meanwhile, the uncompressed baseline–already bottlenecked by storage bandwidth–shows smaller performance degradation, narrowing LILo's throughput gain to 1.8–1.9×.

Evaluation on additional models. Figure 13 presents the inference throughput of LILO and the uncompressed baseline for additional models, Qwen3-235B and OPT-175B, under a 256 GB system memory. For Qwen3-235B, LILO achieves 1.8–2.6× higher throughput, with consistent improvements across batch sizes. For OPT-175B, LILO achieves 1.9–7.0× higher throughput than the baseline, with gains more pronounced at smaller batch sizes and diminishing at $B = 64$. This is because, OPT's standard multi-head attention leads to much more rapid growth in KV cache size with batch size than other models which employ grouped query attention or multi-head latent attention, reducing LILo's benefit.

C. Ablation Study

Table V presents the throughput (tokens/s) of both Llama3-405B and DeepSeek-R1 inference without storage-offloading under four configurations for ablation study: (1) All optimizations; (2) No overlapping between computation and compression; (3) No pipelining between IAA and AVX during decompression; (4) No selective compression in DeepSeek-R1; and (5) No IAA/AVX acceleration, where decompression is performed entirely on CPU cores.[3] Experiments are conducted for content generation category and batch size of 1.

Overlapping decompression and computation achieves a 1.9× throughput gain for DeepSeek-R1, while 1.1× for Llama3-405B. The larger gain in DeepSeek-R1 is attributed to its model architecture, which consists of many small sublayers that benefit more from preserved temporal locality enabled by overlapping (§IV-D). IAA and AVX pipelining result in a throughput gain of 1.3× and 1.6× for Llama3-405B and DeepSeek-R1, respectively, which the improvements align closely with the throughput gains illustrated in Figure 9. Selective compression delivers 1.2× throughput increase for DeepSeek-R1, as its dense part significantly contributes to inference latency, while remaining uncompressed. Finally, no IAA and AVX acceleration results in 6.2× and 6.4× longer latency for Llama3-405B and DeepSeek-R1, respectively.

[3]The throughput is estimated by adding the decompression overhead—computed using the CPU-based decompression throughput reported in Table II—to the baseline latency measured without compression.

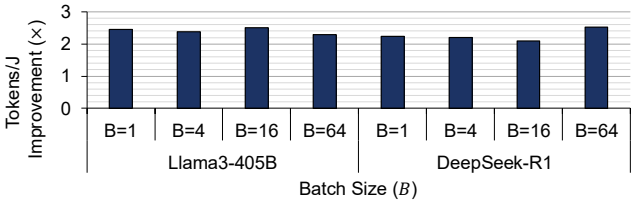


Fig. 14. Tokens/J improvement with LILO over the uncompressed baseline for Llama3-405B and DeepSeek-R1, under 512 GB and 1 TB memory, respectively, for content generation and batch size (B) swept from 1 to 64.

D. Sensitivity Study

Table VI shows the sensitivity of LILo's inference throughput to the ratio of CPU cores allocated to AMX compute threads, AVX BF16 reconstruction threads, and the IAA daemon thread and the IAA chunk size, measured on Llama3-405B with 128/256 input/output lengths (content generation) and batch sizes of 1 and 64. This shows that LILO achieves the highest throughput when using a chunk size of 1 MB and a core allocation of (64, 63, 1) for AMX, AVX, and IAA threads, respectively. Assigning more cores to either compute or reconstruction creates pipeline imbalance between decompression and computation, reducing throughput by 14–37%. Varying the IAA chunk size also impacts decompression throughput, as analyzed in Figure 9, leading to a 4–10% reduction in overall inference throughput.

E. Energy Efficiency Improvement

Figure 14 presents the energy-efficiency improvement of LILO over the uncompressed baseline. The total energy is

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

obtained by summing two separately measured components: (1) energy for compute and decompression, derived by multiplying the average power measured using turbostat [16] with the corresponding latency, and (2) energy for storage access, obtained in the same manner. Overall, LILO consistently improves energy efficiency by 2.1–2.5× across different batch sizes for both models. Note that the energy-efficiency improvement of LILO is smaller than the latency or throughput improvement since the much lower (2–3×) power during storage access reduces its contribution to total energy.

VI. Discussion

The impact of storage prefetching. HuggingFace Accelerate currently lacks storage prefetching (overlapping storage access with compute/decompression). Adding prefetching could reduce storage-offloading overhead for both LILO and the uncompressed baseline, although the benefits differ by scenario. For Llama3-405B under 512 GB, LILO gains 1.6–2.0× throughput with prefetching, benefiting from balanced compute/decompression and storage access latencies. On the other hand, the baseline improves only 1.04–1.2× due to its heavier storage bottleneck. Conversely, for DeepSeek-R1 under 1 TB, only the baseline benefits (1.1–1.2×) since LILO operates fully in memory.

Extension to KV cache compression. For inference with extremely long context windows (*e.g.,* 1M tokens [14]), KV cache can dominate the total memory footprint, reducing LILo's relative benefit in memory reduction and thus its latency and throughput gains. For example, under a 512 GB memory, LILo's performance gain over the uncompressed baseline for Llama3-405B decreases to 1.4× and 1.01× at a 1M token length for batch sizes of 1 and 64, respectively. To address this limitation, LILo's approach can be extended to compress the dynamically generated KV cache using with IAA in runtime. We observe that the KV cache also achieves around 70% compression ratio, similar to model parameters. However, IAA compression throughput only achieves 13 GB/s, which becomes a bottleneck. To mitigate this bottleneck, we can overlap compression with computation as LILO does, leveraging the fact that most KV cache is produced during the prefill stage, which requires long compute time, while only a small fraction is appended per decoding step.

Cost/performance comparison to GPU systems. We compare LILo's hardware cost, power, throughput, and cost efficiency (tokens/$) for DeepSeek-R1 inference under 1 TB system memory against a high-end multi-GPU system composed of three DGX-H100 servers that accommodates DeepSeek-R1's memory footprint, where the GPU system performance is estimated with LLMSimulator [10], [54]. While the DGX system achieves substantially higher raw throughput (55–1.4k tokens/s) than LILO (0.7–4.5 tokens/s), the gap narrows when considering the tokens-per-dollar metric.[4] LILO achieves 3.5k–19.2k tokens/$, compared to the DGX system's 8.1k–75k tokens/$, corresponding to 27–46% of the DGX system's efficiency. This is because LILO incurs significantly lower hardware cost (15.6 K$) [8], [9] and power (564W), compared to the DGX system's cost (1.2 M$) [12] and power (16.8 kW). Despite lagging behind these metrics, LILO offers strong practical advantages by drastically lowering deployment costs (80×) or even zero extra capital expenditure by leveraging servers that hyperscalers have already deployed (§II-A).

[4]The hardware cost is assumed to be amortized across the time period of 3 years and the power consumption is estimated with each system's TDP. Electricity cost of $0.17/kWh is assumed, the national average in the U.S.

VII. Related Work

Model quantization. Quantization-aware training [23], [41] and calibration-based Post-Training Quantization methods [30], [52] have been shown to effectively mitigate accuracy degradation. However, recent studies highlight that beyond preserving accuracy, quantization can introduce new challenges, including increased vulnerability to adversarial attacks [29], unintended bias and toxicity in generation [53], the re-emergence of previously unlearned problematic behaviors [55], and overall reductions in model trustworthiness [34].

Inference with lossless compression. Recent research has explored lossless parameter compression to reduce the memory footprint of convolutional neural networks (CNNs) while preserving model behavior. Eyeriss [24], [25] adopts a run-length compression algorithm and designs a custom dataflow architecture to exploit sparsity, reduce energy consumption and memory bandwidth during CNN inference. Lane-compression [39] groups bits at the same positions into multiple bit lanes and applies the optimal compression algorithm for each lane. A decompressor designed on reconfigurable hardware parallelizes lane-wise decoding.

VIII. Conclusion

In this work, we introduce LILO, a framework that accelerates LLM inference under memory capacity constraints by reducing the storage-offloaded data through lossless compression and on-the-fly decompression during runtime. To achieve high-throughput decompression, LILO leverages Intel CPU's on-chip accelerators including In-Memory Analytics Accelerator (IAA) and Advanced Vector Extensions (AVX) while coordinating their execution with inference computation on Advanced Matrix Extensions (AMX) to ensure efficient parallelism. Our evaluation demonstrates that LILO reduces inference latency by up to 4.9× compared to uncompressed CPU inference under memory capacity constraints, accelerating the deployment of LLMs within tight memory budgets.

Acknowledgments

This research was supported in part by grants from the IBM-Illinois Discovery Accelerator Institute (IIDAI) and the MSIT (Ministry of Science, ICT), Korea (RS-2024-00456287 and RS-2025 02214649) supervised by the IITP (Institute for Information & Communications Technology Planning & Evaluation) and by a generous gift from Intel Corporation. Nam Sung Kim is the corresponding author.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] "Source coding algorithms for fast data compression, author=Pasco, Richard Clark," Ph.D. dissertation, Stanford University CA, 1976.

[2] "HuggingFace model list for text generation," Accessed in 2025. [Online]. Available: https://huggingface.co/models?pipeline_tag=text-generation

[3] "Intel® 64 and IA-32 Architectures Optimization Reference Manual: Volume 1," Accessed in 2025. [Online]. Available: https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&cad=rja&uact=8&ved=2ahUKEwjc1ryy5N2OAxWbLtAFHUUyNOUQFnoECBcQAQ&url=https%3A%2F%2Fcdrdv2-public.intel.com%2F671488%2F248966-Software-Optimization-Manual-V1-048.pdf&usg=AOvVaw2OB6hxnkyssvqf2MIJgDFb&opi=89978449

[4] "Intel® Architecture Instruction Set Extensions and Future Features," Accessed in 2025. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/774990/intel-architecture-instruction-set-extensions-programming-reference.html

[5] "Intel® Extension for PyTorch," Accessed in 2025. [Online]. Available: https://github.com/intel/intel-extension-for-pytorch

[6] "Intel® In-Memory Analytics Accelerator (Intel® IAA)," Accessed in 2025. [Online]. Available: https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/in-memory-analytics-accelerator.html

[7] "Intel® Query Processing Library (Intel® QPL)," Accessed in 2025. [Online]. Available: https://github.com/intel/qpl

[8] "Intel® Xeon® 6980P Processor," Accessed in 2025. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/240777/intel-xeon-6980p-processor-504m-cache-2-00-ghz/specifications.html

[9] "July 2025 Server Memory Prices," Accessed in 2025. [Online]. Available: https://memory.net/memory-prices/

[10] "LLMSimulator," Accessed in 2025. [Online]. Available: https://github.com/scale-snu/LLMSimulator

[11] "LZ4 Bindings for Python," Accessed in 2025. [Online]. Available: https://pypi.org/project/lz4/

[12] "NVIDIA H100 Price Guide 2025: Detailed Costs, Comparisons & Expert Insights," Accessed in 2025. [Online]. Available: https://docs.jarvislabs.ai/blog/h100-price/

[13] "OpenMP," Accessed in 2025. [Online]. Available: https://www.openmp.org/

[14] "Our next-generation model: Gemini 1.5," Accessed in 2025. [Online]. Available: https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/?utm_source=chatgpt.com#architecturel

[15] "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," Accessed in 2025. [Online]. Available: https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[16] "turbostat - Report processor frequency and idle statistics at Linux.org," Accessed in 2025. [Online]. Available: https://www.linux.org/docs/man8/turbostat.html

[17] "zlib — Compression compatible with gzip," Accessed in 2025. [Online]. Available: https://docs.python.org/3/library/zlib.html

[18] B. Abali, B. Blaner, J. Reilly, M. Klein, A. Mishra, C. B. Agricola, B. Sendir, A. Buyuktosunoglu, C. Jacobi, W. J. Starke, H. Myneni, and C. Wang, "Data Compression Accelerator on IBM POWER9 and z15 Processors : Industrial Product," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 1–14.

[19] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022, pp. 1–15.

[20] M. Barot and J. A. de la Pena, "Estimating the size of a union of random subsets of fixed cardinality," *Elemente der Mathematik*, vol. 56, no. 4, pp. 163–169, 2001.

[21] A. Blogs, "Unlocking Enterprise AI: Why are CPUs the Backbone!" 2025. [Online]. Available: https://www.amd.com/en/blogs/2025/unlocking-enterprise-ai-why-are-cpus-the-backbone.html

[22] I. Burstein, "Nvidia data center processing unit (dpu) architecture," in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–20.

[23] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo, "Efficientqat: Efficient quantization-aware training for large language models," *arXiv preprint arXiv:2407.11062*, 2024.

[24] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[25] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.

[26] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[27] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2412.19437

[28] P. Deutsch, "DEFLATE compressed data format specification version 1.3," Tech. Rep., 1996.

[29] K. Egashira, M. Vero, R. Staab, J. He, and M. Vechev, "Exploiting llm quantization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 41 709–41 732, 2025.

[30] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

[31] S. Go and D. Mahajan, "Moetuner: Optimized mixture of expert serving with balanced expert placement and token routing," *arXiv preprint arXiv:2502.06643*, 2025.

[32] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The Llama 3 Herd of Models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[33] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, "Accelerate: Training and inference at scale made simple, efficient and adaptable," 2022. [Online]. Available: https://github.com/huggingface/accelerate

[34] J. Hong, J. Duan, C. Zhang, Z. Li, C. Xie, K. Lieberman, J. Diffenderfer, B. Bartoldson, A. Jaiswal, K. Xu, B. Kailkhura, D. Hendrycks, D. Song, Z. Wang, and B. Li, "Decoding Compressed Trust: Scrutinizing the Trustworthiness of Efficient LLMs Under Compression," 2024. [Online]. Available: https://arxiv.org/abs/2403.15447

[35] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[36] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," 2024. [Online]. Available: https://arxiv.org/abs/2401.04088

[37] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020.

[38] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, and N. S. Kim, "LIA: A Single-GPU LLM Inference Acceleration with Cooperative AMX-Enabled CPU-GPU Computation and CXL Offloading," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 544–558.

[39] Y. Ko, A. Chadwick, D. Bates, and R. Mullins, "Lane compression: A lightweight lossless compression method for machine learning on embedded systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 2, pp. 1–26, 2021.

[40] W. Lian, B. Goodson, E. Pentland, A. Cook, C. Vong, and "Teknium", "OpenOrca: An Open Dataset of GPT Augmented FLAN Reasoning Traces," 2023. [Online]. Available: https://https://huggingface.co/datasets/Open-Orca/OpenOrca

[41] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "Llm-qat: Data-free quantization aware training for large language models," *arXiv preprint arXiv:2305.17888*, 2023.

[42] lz4, "lz4," accessed in 2025. [Online]. Available: https://github.com/lz4/lz4

[43] K. Marchisio, S. Dash, H. Chen, D. Aumiller, A. Üstün, S. Hooker, and S. Ruder, "How does quantization affect multilingual LLMs?" *arXiv preprint arXiv:2407.03211*, 2024.

[44] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier,

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, "GPT-4 Technical Report," 2024. [Online]. Available: https://arxiv.org/abs/2303.08774

[45] Y. Pan, Z. Xia, P.-K. Hsu, L. Hu, H. Kim, J. Sharda, M. Zhou, N. S. Kim, S. Yu, T. Rosing, and M. Kang, "Stratum: System-Hardware Co-Design with Tiered Monolithic 3D-Stackable DRAM for Efficient MoE Serving," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture®*, 2025, pp. 1–17.

[46] L. Press, "AI Inferencing on Intel CPU-Powered Lenovo Servers: Strategic CPU Selection and Implementation," 2025. [Online]. Available: https://lenovopress.lenovo.com/lp2204-ai-inferencing-on-intel-cpu-powered-lenovo-servers-strategic-cpu-selection

[47] I. M. Research, "Server Memory for Data Centers Market Growth Analysis, Dynamics, Key Players and Innovations, Outlook and Forecast 2025-2032," 2025. [Online]. Available: https://www.intelmarketresearch.com/server-memory-for-data-centers-2025-2032-198-6072

[48] A. H. Robinson and C. Cherry, "Results of a prototype television bandwidth compression scheme," *Proceedings of the IEEE*, vol. 55, no. 3, pp. 356–364, 2005.

[49] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023, pp. 31 094–31 116.

[50] Y. Song, Z. Mi, H. Xie, and H. Chen, "Powerinfer: Fast large language model serving with a consumer-grade gpu," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 590–606.

[51] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 06, pp. 8–19, 1984.

[52] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.

[53] Z. Xu, A. Gupta, T. Li, O. Bentham, and V. Srikumar, "Beyond perplexity: Multi-dimensional safety evaluation of llm compression," *arXiv preprint arXiv:2407.04965*, 2024.

[54] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. Ahn, "Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching," in *MICRO*, 2024.

[55] Z. Zhang, F. Wang, X. Li, Z. Wu, X. Tang, H. Liu, Q. He, W. Yin, and S. Wang, "Catastrophic Failure of LLM Unlearning via Quantization," *arXiv preprint arXiv:2410.16454*, 2024.

[56] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Transactions on information theory*, vol. 23, no. 3, pp. 337–343, 2003.

[57] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 2003.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

APPENDIX

A. Abstract

This artifact appendix provides a guideline for using LILO for compressed LLM inference acceleration and how to reproduce the three key results of this paper: 1) LILO's inference throughput comparison to the uncompressed baseline 2) inference latency breakdown of LILO and the uncompressed baseline, and 3) throughput improvement with LILO over the uncompressed baseline across varying memory capacity. The following subsections outline the steps to access, setup the software environment, and to run experiments with LILO on a CPU system with Advanced Vector Extension (AVX), Advanced Matrix Extension (AMX), and In-memory Analytics (IAA) accelerators.

B. Artifact check-list (meta-information)

- **Model:** Llama3-405B and DeepSeek-R1
- **Run-time environment:** Ubuntu 22.04.4 LTS, Linux kernel 6.8.
- **Hardware:** Intel Xeon 6980P Processor, Micron 7450 NVMe M.2 SSD.
- **Metrics:** Inference latency (seconds) and throughput (tokens/s).
- **Output:** .log files containing inference latency measurements
- **Experiments:** LLM inference with LILO and the uncompressed baseline to reproduce Figures 11 and 12.
- **How much disk space required (approximately)?:** 720 GB for the model weights and the Docker images.
- **How much time is needed to prepare workflow (approximately)?:** 1 hour.
- **How much time is needed to complete experiments (approximately)?:** 20 hours.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** Apache-2.0 license.
- **Archived:** https://doi.org/10.5281/zenodo.17862931

C. Description

1) How to Access.: The scripts and guidelines for the deployment of LILO are publicly available on GitHub (https://github.com/ece-fast-lab/HPCA-2026-LILo) and Zenodo (https://doi.org/10.5281/zenodo.17862931. The repository contains step-by-step scripts under `scripts/` for environment preparation, baseline evaluation, decompression-based inference, and storage-offloading characterization, as well as a separate figure generation module.

2) Hardware Dependencies.: A server with an Intel Xeon Scalable Processor ($\geq$ 4th generation) equipped with at least one IAA is required. For best performance reproduction, four IAAs are recommended. The following BIOS settings must be enabled:

- Hardware prefetch
- LLC prefetch
- Adjacent cache prefetch

3) Software Dependencies.: We provide pre-built docker images that contain all required runtime dependencies and libraries, including:

- Ubuntu 22.04 LTS
- Linux kernel 6.8.0-49-generic
- GCC 13.1.0
- Intel Query Processing Library (QPL)
- Intel `idxd-config` for IAA configuration
- Intel Extension for PyTorch (IPEX)

Note that the usage of IAA requires the installation of the Intel Query Processing Library (QPL) (https://github.com/intel/qpl) and `idxd-config` (https://github.com/intel/idxd-config), both of which are already incorporated into the provided docker images. To enable IAA, IOMMU also should be enabled via the following GRUB setting:

```
GRUB_CMDLINE_LINUX="quiet iommu=pt
    intel_iommu=on sm_on no5lvl splash
    intel_pstate=disable efi=nosoftreserve
    nokaslr"
```

D. Installation

First, download the Github repository as follows:

```
$ git clone https://github.com/ece-fast-lab/\
   HPCA-2026-LILo.git
$ cd HPCA-2026-LILo
```

The full reproduction pipeline is organized into four steps under `scripts/`. Users should follow the steps sequentially.

E. Experiment Flow

1) Step 0: Environment Setup: To prepare the software environment and system configuration, run the following commands:

```
$ cd scripts/step_0_env_setup
$ bash ./get_docker.sh
$ bash ./env_setup.sh
```

These commands download all required Docker images, fix the CPU frequency, and configure the IAA devices using `idxd-config`. The repository further provides instructions creating a cropped version of Llama3-405B and DeepSeek-R1 and randomizing MoE routing for DeepSeek-R1 with a patch.

2) Step 1: Uncompressed Baseline Inference: To collect inference latency for the uncompressed baseline, run the following command:

```
$ cd ../step_1_baseline
$ bash ./baseline.sh <docker_name>
```

This script launches a Docker container, executes the uncompressed baseline inference for both Llama and DeepSeek models, and saves the latency logs under the `results` directory. Note that you must update the mounted directory (`-v`) in `baseline.sh` to point to the storage location of your model weights on the local machine; the same update is also required for Steps 2 and 3.

*3) Step 2: Inference with Decompression (*LILO*):* To evaluate inference with on-the-fly decompression using LILO, run the following command:

```
$ cd ../step_2_decomp
$ bash ./decomp.sh <docker_name>
```

This script runs decompression-enabled inference for both Llama and DeepSeek models and writes the resulting latency logs to the `results` directory.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.

4) Step 3: Storage-Offloading Characterization: To characterize the overhead of storage-offloaded inference, run the following command:

```
$ cd ../step_3_storage_offload
$ bash ./storage_offload.sh <docker_name>
```

This step evaluates storage-offloaded inference using HuggingFace Accelerate by sweeping different amounts of offloaded data and recording the corresponding latency results. These measurements are later used to characterize the storage-offloading overhead by fitting a performance model, which is then combined with the latency results of both LILO and the uncompressed baseline.

F. Evaluation and Expected Results

To reproduce the key experimental results from the paper, the collected data is used to generate (1) Figure 11(a), which compares the inference throughput between LILO and the baseline, (2) Figure 11(b), which presents the latency breakdown, and (3) Figure 12, which shows throughput improvement under varying memory capacities. To generate all figures automatically, run the following command:

```
$ cd ../../fig_gen
$ python3 generate_figures.py
```

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on May 04,2026 at 04:27:51 UTC from IEEE Xplore. Restrictions apply.